Life Expectancy Intelligence Dashboard

End-to-End Machine Learning Web Application

Author	Srijan Basu
Degree	MCA (Master of Computer Applications)
Project Type	End-to-End ML Portfolio Project
Live URL	https://life-expectancy-prediction-2026.streamlit.app
GitHub	https://github.com/BasuSrijan2003/life-expectancy-dashboard
Dataset	WHO Life Expectancy Dataset — Kaggle
Model	Random Forest Regressor — R2 Score: 96.91%

1. Project Overview

This project is a complete, production-deployed machine learning web application that predicts and analyses global life expectancy. It was built entirely from scratch — from raw data cleaning to a live public URL — demonstrating the full software development lifecycle of an ML project.

The application allows any user — without knowledge of Python or data science — to interact with a trained Random Forest model through a browser. Users can explore real WHO health data, get AI predictions, run what-if simulations, and query the dataset using plain English questions.

What makes this a portfolio-grade project:

- Not just a notebook — it is a deployed, shareable web application with a permanent URL.
- Covers the complete ML pipeline: data cleaning, feature engineering, model training, serialization, and inference.
- Built a professional UI with 5 interactive tabs using only Python — no HTML, CSS, or JavaScript written manually.
- Implemented a smart data query engine (Ask the Data) using pure pandas logic — no external AI API.
- Deployed for free on Streamlit Community Cloud with auto-training on cold start.

2. Technology Stack

Category	Technology	Purpose
Machine Learning	Scikit-learn	Random Forest model training and prediction
Data Processing	Pandas, NumPy	Data cleaning, transformation, feature engineering
Model Persistence	Joblib	Saving and loading the trained model (.pkl file)
Web Framework	Streamlit	Building the entire interactive web UI in Python
Visualisation	Plotly	Interactive charts — gauges, radar, bar, heatmap, line
Deployment	Streamlit Community Cloud	Free cloud hosting with permanent public URL
Version Control	Git + GitHub	Source code management and deployment trigger
Language	Python 3.x	Entire project — data, model, and UI — in one language

3. Dataset

Source: WHO Life Expectancy Dataset — downloaded from Kaggle.

Note: The dataset is not committed to the public GitHub repository as per Kaggle's data sharing guidelines. It is stored locally and in the deployment environment only.

Property	Value
Total Records	2,938 rows
Total Features	22 columns
Countries Covered	193
Time Period	2000 to 2015 (16 years)
Target Variable	Life expectancy (years)
Missing Values (raw)	2,563 cells across various columns
Missing Value Strategy	Filled with column mean

Features used in the model (after dropping redundant columns):

Year, Status (Developed/Developing), Adult Mortality, Alcohol, Percentage Expenditure, Hepatitis B, Measles, BMI, Under-five Deaths, Polio, Total Expenditure, Diphtheria, HIV/AIDS, GDP, Population, Thinness 1-19 years, Income Composition of Resources, Schooling.

Dropped columns: Country (text identifier), Infant Deaths (redundant with under-five deaths), Thinness 5-9 years (redundant with thinness 1-19 years).

4. Machine Learning Model

4.1 Algorithm: Random Forest Regressor

A Random Forest is an ensemble learning method that builds multiple decision trees during training and outputs the average prediction of all trees. It was chosen because:

- Handles missing data well and is robust to outliers.
- Does not require feature scaling — no StandardScaler needed.
- Naturally provides feature importance scores.
- Resistant to overfitting due to the ensemble of 100 independent trees.
- Achieves high accuracy on tabular health data without hyperparameter tuning.

4.2 Training Configuration

Parameter	Value
Algorithm	RandomForestRegressor (Scikit-learn)
Number of Trees	100 (n_estimators=100)
Random State	42 (for reproducibility)
Parallel Jobs	-1 (uses all CPU cores)
Train/Test Split	80% training, 20% testing
Split Random State	42

4.3 Model Performance

Metric	Value	Meaning
R2 Score	96.91%	The model explains 96.91% of all variation in life expectancy
Mean Absolute Error	1.05 years	On average, predictions are off by just 1.05 years
Training Samples	2,350	80% of 2,938 records used for training
Test Samples	588	20% of 2,938 records used for evaluation

4.4 Top Feature Importances

The Random Forest model internally ranks how much each feature contributes to predictions:

Rank	Feature	Importance	Interpretation
1	HIV/AIDS	59.43%	Single most powerful predictor of lifespan
2	Adult Mortality	16.05%	Direct measure of healthcare system quality
3	Income Composition	14.39%	Wealth distribution index (HDI component)
4	Schooling	1.86%	Years of education strongly linked to health outcomes
5	BMI	1.46%	Average body mass index of the population

5. Project File Structure

The project follows a clean, professional folder structure that separates concerns: data, model artifacts, and application code are all in distinct locations.

```
life-expectancy-dashboard/ ■■■ app.py ← Main Streamlit web application (5 tabs) ■■■ train_model.py ← Standalone model training script ■■■ requirements.txt ← Python package dependencies ■■■ model/ ■ ■■■ rf_model.pkl ← Serialized trained model (frozen brain) ■ ■■■ feature_names.json ← Exact column order model expects ■ ■■■ feature_stats.json ← Min/max/mean for each feature (slider ranges) ■■■ data/ ■■■ Life Expectancy Data.csv ← WHO dataset (not in GitHub)
```

5.1 train_model.py — The Training Script

This script is run once locally. It loads the raw CSV, cleans the data, trains the Random Forest, evaluates it, and saves three output files into the model/ folder. After this script runs, the app never needs to read the CSV again to make predictions — it loads the .pkl file directly, which is milliseconds fast.

Steps inside train_model.py:

- Load CSV and strip hidden whitespace from column names
- Encode Status column: Developing=0, Developed=1
- Drop Country, infant deaths, thinness 5-9 years
- Fill missing values with column mean
- Split into 80/20 train/test sets with random_state=42
- Train RandomForestRegressor(n_estimators=100)
- Evaluate: print R2 Score and Mean Absolute Error
- Save rf_model.pkl using joblib.dump()
- Save feature_names.json — list of 18 feature names in exact training order
- Save feature_stats.json — min, max, mean for each feature

5.2 model/rf_model.pkl — The Frozen Brain

This is the serialized trained model. Joblib serializes the entire RandomForest object (all 100 decision trees, their splits, thresholds, and leaf values) into a binary file. When the app loads it with joblib.load(), the model is instantly ready to predict — no retraining needed. File size: approximately 19MB.

5.3 model/feature_names.json — The Safety Net

This file stores the exact list of 18 feature column names in the exact order the model was trained on. When building a prediction input from user slider values, the app constructs a DataFrame using this list as the column order. This prevents silent column mismatch bugs that would corrupt predictions without any error message.

5.4 model/feature_stats.json — Slider Configuration

Stores the minimum, maximum, and mean value for each of the 18 features calculated from the full dataset. The Streamlit sliders use these values as their range bounds and default starting position, ensuring sliders are always calibrated to real-world data.

6. Application — 5 Interactive Tabs

The entire application is built in a single file: app.py. It uses Streamlit's tab component to organize 5 distinct features. The model and data are loaded once at startup using @st.cache_resource and @st.cache_data decorators — this means no matter how many times a user interacts with the app, the model is only loaded from disk once.

Tab 1 — Country Explorer

User selects any of 193 countries and any year from 2000 to 2015 using two dropdowns. The app queries the raw dataset for that row, builds an input dictionary, and calls model.predict() to get the AI prediction. The tab displays:

- Gauge chart showing predicted life expectancy (green >75, yellow 60-75, red <60)
- Actual recorded life expectancy from WHO data with delta vs predicted
- Key stats: Schooling years, HIV/AIDS rate, GDP per capita
- Radar chart comparing 6 health dimensions of the country vs global average
- Line chart showing the country's life expectancy trend from 2000 to 2015 with selected year marked

Tab 2 — AI Predictor

User builds a completely hypothetical country by adjusting 18 sliders across 4 categories: Country Profile, Health and Disease, Vaccination Coverage, and Socioeconomic. All slider ranges are set from feature_stats.json (real data bounds). On clicking Predict, the app assembles the values into a DataFrame in the exact column order from feature_names.json and calls model.predict(). Output includes:

- Large number display showing predicted years
- Gauge chart with colour coding
- Delta card showing difference from global average of 69.2 years
- Grouped bar chart comparing user inputs against global averages for 6 key features

Tab 3 — What-If Simulator

The most analytically powerful tab. User selects a real country as a baseline. The app loads that country's actual data and makes a baseline prediction. The user then adjusts 6 specific levers (Schooling, HIV/AIDS, GDP, Adult Mortality, Income Index, Health Expenditure). On clicking Run Simulation:

- Two side-by-side gauge charts: Before and After
- A large delta box showing how many years were gained or lost
- Impact breakdown bar chart: each lever is isolated and tested individually to show its individual contribution

The impact breakdown works by running 6 separate predictions — one for each lever — changing only that one feature while keeping all others at the baseline value. This is a simplified version of the SHAP (SHapley Additive exPlanations) concept applied manually using pandas.

Tab 4 — Feature Importance

Displays the internal feature importance scores extracted directly from the trained Random Forest using model.feature_importances_. These scores represent how much each feature reduces impurity across all 100 trees. Tab includes:

- Horizontal bar chart of all 18 features sorted by importance with percentage labels
- Three insight cards explaining the top 3 factors in plain English
- Full interactive correlation heatmap of all numeric features using Plotly imshow

Tab 5 — Ask the Data

A natural language query interface built without any AI API — pure Python and pandas. The user types a question in plain English and the engine parses it using keyword matching and regex to identify the intent, country, year, and metric, then runs the appropriate pandas query.

Query types supported:

Question Pattern	Engine Action
Which country had the highest X in YEAR?	Filters by year, finds idxmax() for that column
Which country had the lowest X?	Finds idxmin() globally or filtered by year
What is the X of COUNTRY in YEAR?	Direct row lookup and value extraction
Compare COUNTRY1 and COUNTRY2 in YEAR	Fetches both rows, computes difference
Which country improved the most?	Merges 2000 and 2015 data, calculates delta
What factors affect life expectancy?	Returns top 5 from model.feature_importances_
Developed vs developing countries	Groups by Status, computes mean life expectancy
Average life expectancy in YEAR?	Global mean for that year from raw data

7. Deployment Architecture

The app is deployed on Streamlit Community Cloud — a free hosting platform for Streamlit apps connected to public GitHub repositories.

7.1 Deployment Flow

- Code is pushed to GitHub repository: BasuSrijan2003/life-expectancy-dashboard
- Streamlit Cloud detects the push and automatically triggers a redeploy
- Cloud server installs dependencies from requirements.txt
- On first boot, app.py checks if model/rf_model.pkl exists
- If not found (cold start), the auto-train block runs — trains model in ~30 seconds
- Model is saved to the ephemeral filesystem for the session
- App loads model into memory using @st.cache_resource — stays loaded for all users
- App is live at the permanent public URL

7.2 Cold Start Auto-Training

Because the model file (19MB) exceeds the practical size for GitHub free plans, it is not committed to the repository. Instead, the app contains a cold-start block at the very top of app.py that checks for the model file and trains it from scratch if missing. This runs only once per deployment session and takes approximately 30 seconds. All subsequent user interactions use the cached model and are instant.

7.3 Caching Strategy

Decorator	Applied To	Effect
@st.cache_resource	load_model()	Model loaded from disk exactly once, shared across all users
@st.cache_data	load_data()	CSV loaded once, 2938 rows available instantly
@st.cache_data	load_meta()	JSON files loaded once, feature names and stats cached

8. Resume and LinkedIn Presentation

8.1 Resume Entry

Project Title: Life Expectancy Intelligence Dashboard

One-line description for resume header:

End-to-end ML web application predicting life expectancy for 193 countries using a 96.91% accurate Random Forest model, deployed live with Streamlit.

Bullet points for resume (pick 4-5):

- • Built a complete ML pipeline from raw WHO data to a live web application — data cleaning, feature engineering, model training, serialization, and cloud deployment.
- • Trained a Random Forest Regressor (100 trees, Scikit-learn) on 2,938 records achieving R2=96.91% and MAE=1.05 years on unseen test data.
- • Developed a 5-tab interactive dashboard using Streamlit and Plotly featuring country exploration, real-time AI prediction, what-if simulation, feature importance analysis, and a natural language data query engine.
- • Implemented model persistence using Joblib and a cold-start auto-training mechanism for cloud deployment on Streamlit Community Cloud.
- • Built a keyword-based NLP query engine using pure Pandas that answers plain-English questions about the dataset without any external AI API.
- • Managed full project lifecycle using Git and GitHub — version control, remote repository, and continuous deployment.

Tech stack line for resume:

Python, Scikit-learn, Pandas, NumPy, Streamlit, Plotly, Joblib, Git, GitHub, Streamlit Cloud

8.2 LinkedIn Post

Suggested LinkedIn post text:

Excited to share my latest project — deployed and live!

I built the Life Expectancy Intelligence Dashboard — a complete, end-to-end machine learning web application that predicts life expectancy for 193 countries using WHO health data.

What I built:
- Trained a Random Forest model (96.91% R2 accuracy) on 2,938 WHO records
- Built a 5-tab interactive dashboard with country exploration, real-time AI prediction, what-if scenario simulation, feature importance visualization, and a plain-English data query engine
- Deployed it live for free using Streamlit Community Cloud

Tech used: Python, Scikit-learn, Pandas, Streamlit, Plotly, Joblib, Git

The most interesting insight: HIV/AIDS rate alone accounts for 59.4% of the model's prediction power — far more than GDP or schooling.

Live app: https://life-expectancy-prediction-2026.streamlit.app

GitHub: https://github.com/BasuSrijan2003/life-expectancy-dashboard

#MachineLearning #Python #Streamlit #DataScience #MCA #Portfolio #RandomForest

9. Key Engineering Decisions

Why Random Forest and not Linear Regression?

Random Forest handles non-linear relationships between features and the target, does not require feature scaling, provides built-in feature importance, and is naturally resistant to overfitting. Linear Regression achieved ~75% R2 on this dataset vs 96.91% for Random Forest.

Why Joblib and not Pickle?

Joblib is recommended by Scikit-learn for serializing models that contain large NumPy arrays (as decision trees do). It is faster and more memory efficient than Python's built-in pickle for this use case.

Why store feature_names.json separately?

When a user submits slider values, the app builds a DataFrame and calls model.predict(). If the columns are in the wrong order, the model silently produces wrong predictions with no error. Storing the exact training column order and enforcing it at inference time is a standard ML engineering safety practice.

Why @st.cache_resource for the model?

Without caching, the 19MB model would be loaded from disk on every single user interaction. cache_resource tells Streamlit to load the model once and share it across all sessions and all users — making predictions instant.

Why no external AI API for the query tab?

This is an ML project, not a GenAI project. The query engine demonstrates the ability to extract meaningful insights from structured data using pandas — a core data science skill. Using an external API would obscure the engineering and increase cost and complexity.